Reflected PSO for Robot Path Planning

Ali Rodan *
King Abdullah II School of Information Technology
The University of Jordan
Amman, Jordan
Email: a.rodan@ju.edu.jo

2024 IEEE International Conference on Evolving and Adaptive Intelligent Systems (EAIS) | 979-8-3503-6623-5/24/$31.00 ©2024 IEEE | DOI: 10.1109/EAIS58494.2024.10570016

***Abstract*—In this paper, we introduce an enhanced Particle Swarm Optimization using reflection strategy called Reflected Particle Swarm Optimization (RPSO) algorithm, RPSO designed to enhance robot path planning in complex environments such as static environment with obstacles. The integration aims to improve the exploration and exploitation capabilities of the search process. RPSO has been compared with state-of-art and well-known optimization algorithms. The results demonstrate that RPSO consistently outperforms the compared algorithms in finding the shortest paths and maintaining high levels of consistency across different trials. Furthermore, the results show that RPSO, with its innovative reflection mechanism, offers a significant advancement in robot path planning.**

***Index Terms*—Particle Swarm Optimization, PSO, Reflected PSO, Robotic, Path Planning, Path Finding**

I. Introduction

Metaheuristics are higher-level procedures that are meant to locate, produce, or search for a solution to an optimization issue that is sufficiently good, particularly when the information is incomplete or ambiguous. Through the utilization of metaheuristic algorithms, metaheuristic optimization addresses optimization issues . When it comes to a wide range of fields, including engineering design, economics, and robot path planning, optimization is practically everywhere. The main purpose of metheuristics is to optimize the usage or parameters of different problems such as time, and other resources, The intelligence of metaheuristics has led to their widespread applications in a variety of optimization issues. [1].

Under a wide variety of difficult constraints, the majority of optimizations problems in the real world are extremely nonlinear or multimodal. Finding an optimal solution in these problems is not always guaranteed, even when dealing with a single objective case. However, searching for a solution that is ideal or even one that is sub-optimal is not a simple task in general [2]. One of the most famous optimizers in the field of metaheuristics is Particle Swarm Optimization (PSO). PSO is a stochastic optimization technique that is based on the concept of decentralized self manged swarm. The PSO algorithm is capable of simulating the social behavior of animals such as the flock of birds. In order to reach best solution, these swarms behave in a cooperative manner, and each individual member of the swarms continues to alter the search pattern in accordance with the experiences it has gained from both its own and other members experiences. [3]

PSO is distinct from previous evolutionary computation methods in that it assigns a velocity and position to each individual particle. Particles move around the search space at velocities that are dynamically changed in accordance with the behaviors and cooperation of all particles. In addition, the particles have a propensity to move in the direction of the search region that is getting better and better as the search process progresses.

Due to the ability of PSO to effectively find solutions in different search spaces, many researchers have used PSO in different domains and applications [4] ever since it was first presented in 1995. However, the field of robot path planning for a mobile robot has gained a lot of attention, and numerous strategies using metaheuristics have been proposed to solve this problem, especially in an area that contains a set of obstacles. One of the main goals of the robot path problem is to locate paths for robots that are free of collisions and have the optimal path between the starting point and the goal point [5].

In this paper, we introduce an enhanced PSO optimizer using the reflection strategy, for solving the robot path planning problem called Reflected Particle Swarm Optimization (RPSO), where the reflection strategy have been proven to be an effective method for enhancing metaheuristic results [6]. RPSO creates new solutions that offer a balance between exploration and exploitation, where the robot dynamically adjust its path according to different environments.

RPSO is proposed to address limitations in standard Particle Swarm Optimization (PSO), especially in robotic path planning scenarios involving complex environments. Standard PSO can struggle with premature convergence to local optima and imbalances in exploration and exploitation. These challenges are significant in environments with both static and dynamic obstacles. RPSO's reflection strategy enhances solution diversity and adaptability, addressing the convergence issue and improving the algorithm's capability in navigating environments with obstacles.

The remainder of the paper is organized as follows: Section II discusses an overview of background about related work in the areas of robot path planning and optimization techniques. In Section III, we present the proposed reflected PSO, its mathematical model, and an algorithm for enhanced robot path planning. Application of RPSO for solving robot path problems is described in Section IV. In Section V, the experimental studies with empirical results are discussed. Finally, our work is concluded in Section VI.

Authorized licensed use limited to: BMS College of Engineering. Downloaded on November 05,2026 at 10:21:27 UTC from IEEE Xplore. Restrictions apply.

II. Related Work

This section presents recent work in the field of robotic path planning using metaheuristics. Li et al. [2] describes a technique that provides an improved version of the Particle Swarm Optimization (IPSO) algorithm that integrates a variety of different tactics, such as: the learning factor of improved control, the cubic spline interpolation function, the exponential damping inertia weight and the uniform distribution. IPSO outperforms traditional PSO algorithms because it requires fewer iteration steps and produces more optimal results. The authors [2] showed how IPSO can improve the efficiency of mobile robot path planning by optimizing the path length and significantly reducing the time required for simulation.

Cultural-Particle Swarm Optimization (C-PSO) [7] is a hybrid evolutionary approach that combines the cultural algorithm with PSO to achieve a balance between exploration and exploitation performance while avoiding the pitfalls of local extremes . C-PSO is characterized by a number of features, including the probabilistic inertia weight update technique based on an improved neighborhood rule

Furthermore, another hybrid approach (Particle Swarm Optimization with Simulated Annealing PSO-SA) was developed for AGV route planning in warehouses [8]. PSO-SA contributed significantly to the advancement of the path planning field, where it provided improved overall performance, flexibility, scalability, and customization of robotic route planning systems. One of the most notable features of the PSO-SA method is its ability to improve the efficiency of detecting the optimal global solution while reducing the risk of becoming trapped in local optimal solutions [8]

On the other hand, an updated technique known as Particle Swarm Optimization combined with Gray Wolf Optimization (PSO-GWO) [9] is presented to improve the traditional PSO path planning method. PSO-GWO incorporates chaos theory as well as a novel adaptive inertia weight to solve difficulties such as premature convergence and insufficient global search capability.

Zhang et al. [10] utilized an Improved Localized Particle Swarm Optimization technique (ILPSO) to address the issues of mobile robot path planning. ILPSO overcomes key flaws in standard PSO, such as local minima, early convergence, and low efficiency. To avoid the algorithm from falling into a local minimum and increase convergence speed, modifications in acceleration factors, inertia weights, and localization were made. Moreover, an extended Gaussian distribution is used to increase particle variety and the smoothing principle.

Ma [11] studied the application of an improved Ant Colony Algorithm in robot path planning. He theoretically and computationally expounds the ant colony algorithm and proposes an improved calculation process for it. Shen and Wang [12] introduced a hybrid intelligent optimization (HIO) algorithm for mobile robot path planning with a two-stage process. The first stage utilizes the Dijkstra algorithm to determine the optimal solution and the dimension of the element in the population for mobile robot path planning. The second stage involves a new screening mechanism using the Improved Five-Elements Cycle Model (IFECM), which divides the population into three groups (top, middle, and low element groups). The PSO algorithm and crossover operator update the middle element group, while the mutation operator updates the low element group. These updated groups are then used to refine the top element group and the global best solution (gbest).

Lastly, A novel swarm intelligence algorithm with a parasitism-relation-based structure for mobile robot path planning (ParaPA), was suggested by Ren et al. [13]. It combines the advantages of the Artificial Bee Colony (ABC) algorithm [14] and the modified Particle Swarm Optimization (MPSO) [15]. This method is built on a multi-swarm evolutionary framework that draws inspiration from the biosphere's parasitic connection, where three steps comprise the evolution process: independent evolution, fusion, and interaction. Further, to increase search speed, the segment obstacle-avoiding technique is used, and two fitness functions evaluate the safety and consumption concerns for path selection.

III. Proposed Reflected Particle swarm optimization (RPSO) for robot path planning

This section begins by discussing the mathematical model behind the proposed method. Following this, the algorithm is presented. Later, we discuss the enhanced Robot Path Planning with Reflection Strategy.

The Reflected Particle Swarm Optimization (RPSO) algorithm is an enhanced metaheuristic designed for solving robot path planning problems in a static enviroment. The core objective is to determine the shortest possible path from a designated start point to an endpoint, while avoiding obstacles [16].

A. Mathematical model

The mathematical model for the Reflected PSO algorithm includes:

1) **Particle Representation**: Each particle's position is represented by two points $x_i(t)$, and $y_i(t)$ in the two-dimensional space as shown in Eq.1 [17]:

$$P_i(t) = \{x_i(t), y_i(t)\}. \tag{1}$$

Where $P_i(t)$ is the position of the ith particle at time t.

2) **Velocity Update**: The velocity of each particle is updated as shown in Eq.2 [17]:

$$\begin{aligned} V_i(t+1) = w \cdot V_i(t) + c_1 \cdot r_1 \cdot (P_{\text{best}_i} - P_i(t)) + \\ c_2 \cdot r_2 \cdot (P_{\text{global_best}} - P_i(t)). \end{aligned} \tag{2}$$

Where $V_i(t+1)$ is the velocity of the ith particle at the iteration time $t+1$, w is the inertia weight, c_1 and c_2 are acceleration coefficients, r_1 and r_2 are random numbers, P_{best_i} is the best position found so far by the ith particle, and $P_{\text{global_best}}$ is the best position found by any particle in the swarm.

3) **Position Update**: The position of every particle in the swarm is calculated as shown in Eq. 3.

$$P_i(t+1) = P_i(t) + V_i(t+1). \tag{3}$$

Authorized licensed use limited to: BMS College of Engineering. Downloaded on November 05,2026 at 10:21:27 UTC from IEEE Xplore. Restrictions apply.

Where, $P_i(t+1)$ is the the position of the ith particle at time $t+1$.

4) **Proposed Reflection Strategy**: Based on the idea of modifying the path that a particle takes within the search space, which was studied by Zhang's work [6]. We propose using the reflection Strategy in PSO as illustrated in Equation 4. The fundamental purpose of reflection is to find new solution in the search space that may have values that is better than the founded ones. However, finding more solutions allows the RPSO to be able to solve more complicated issues, and it is useful in circumstances where particles are confined to local minima.

$$P_{\text{reflected}_i} = (1+\alpha) \cdot P_{\text{global_best}} - \alpha \cdot P_i(t). \quad (4)$$

Where, $P_{\text{reflected}_i}$ denotes the recalculated position of the ith particle following the reflection process. The coefficient α represents the degree of reflection, serving as a pivotal factor in determining the new trajectory. $P_{\text{global_best}}$ is the optimal position identified across the entire swarm up to the current iteration, and $P_i(t)$ represents the position of the ith particle at time t.
This reflective strategy is essential in enhancing the algorithm's ability to navigate through and beyond locally optimal solutions, thereby contributing to the identification of more robust global solutions.

5) **Fitness Evaluation**: The fitness of each particle is evaluated based on the length of the path and violations of constraints. The objective function is defined as

$$f(P_i) = L(P_i) + \beta \cdot V(P_i), \quad (5)$$

where $L(P_i)$ is the length of the path, $V(P_i)$ is the violation measure, and β is a penalty coefficient.

As it can be seen from the mathematical model, each potential solution is represented by a 'particle' in the swarm that has a position $P_i(t)$ (see Eq.1) indicating a distinct point on the potential path. The essence of the RPSO lies in the iterative update of each particle's or the reflected particle if it has better position and velocity. The velocity update (Eq.2) contains three components: the particle's current velocity, its best-known position (the cognitive component), and the global best-known position found by any particle in the swarm (the social component).

In addition, the particle's new position is determined by adding the updated velocity to its current position. The algorithm's novelty is further emphasized through the implementation of a reflection strategy (Eq. 4). Where each particle's position is mirrored relative to the global best position, effectively allowing the exploration of new paths that may yield better solutions. Finally, the particle's position fitness is calculated as shown in Eq. 5. This evaluation process entails a consideration of the path length while simultaneously ensuring the avoidance of any obstacles present within the search domain. The objective of this step is minimize the fitness function, which is essential in identifying the shortest path by avoiding obstacles. RPSO reflection strategy integration significantly increases its effectiveness in robot path planning, making it easier to determine an optimal path.

B. Proposed RPSO Algorithm

The proposed algorithm commences by initializing essential parameters, such as the number of particles, iterations, and dimensions of the search space. It sets the starting and destination points for path planning and identifies obstacles within the environment as coordinate points. Each particle, symbolizing a potential solution path, is assigned an initial position and velocity. Further, RPSO advances into a loop that spans the predetermined number of iterations. Throughout this loop, the positions of the particles are dynamically updated.

Algorithm 1 Path Planning using Reflected PSO

```
Initialize n_particles, n_iterations, n_dimensions, search space
Define start point, destination point, obstacles
Initialize particle positions and velocities
for each iteration do
    for each particle do
        Update particles velocity using Eq. 2
        Update particles position using Eq. 3
        Reflect best particle position according to Eq. 4
        if reflected path is valid and shorter then
            Update particle position to reflected position
        end if
    end for
    Update the optimal best position
end for
Calculate shortest path distance
```

In the RPSO, each particle's velocity is modified based on both its individual learning experience and the collective knowledge gleaned from the best-performing particle in the swarm. This dual approach encourages a thorough exploration of the solution space. After updating its velocity, a particle's new position is found, and where the reflection strategy is applied. Each particle position is reflected across the best particle's position, generating an alternative potential path. This reflected path is then evaluated for its feasibility, ensuring it does not intersect with any obstacles and remains within the boundaries of the search space. However, the location of the particle is changed to reflect the new position when it has a lower value that represents the shortest path. In the initialization step of RPSO, a number of possible solutions within the search space are generated, as well as the coordinates of the starting point and the final destination. Further, the evaluation of the efficiency of each path is calculated using the fitness function, where the efficiency of each path is dependent on its length and its ability to navigate around obstacles. RPSO use Particle Swarm Optimization (PSO) to explore the robot path, where the particles are continuously modifying their trajectories in response to the best solution found at each iteration aiming to find the shortest path. In addition, the reflection improves the iterative process of particle position adjustment, which

Authorized licensed use limited to: BMS College of Engineering. Downloaded on November 05,2026 at 10:21:27 UTC from IEEE Xplore. Restrictions apply.

seeks to achieve a balance between discovering new solutions and taking advantage of founded point, until the algorithm achieves its predetermined end point. Figure 1 illustrates the effect of the reflection strategy in finding the optimal path across challenging and obstacle-filled environments. During the iteration process, RPSO is simultaneously incorporating a reflection technique that enhances its search capabilities. However, RPSO search is followed by the creation of an alternative trajectory as described in Eq. 4. This new route is then evaluated to see whether or not it is better than the founded path. In the event that this reflected point is on the shorter path and also avoids the obstacles, it will be chosen as the new location, therefore presenting an enhanced approach to discovering possibly improved paths. It focus is on ensuring a dynamic balance, continuously moving between discovering new viable paths and optimizing the most promising ones. Iteratively, the trajectories are carefully adjusted by RPSO, where it takes into consideration that the paths are guaranteed to stay practicable and follow environmental restrictions, such as avoiding barriers and obstacles. This methodology emphasizes how RPSO may effectively resolve path planning issues in difficult situations.

IV. Application of RPSO for solving Robot Path problems

This section presents the application of RPSO in solving robot path planning in a static environment with obstacles, allowing the robot to move towards the goal point while avoiding obstacles.

A. Path Planning in Static Environment

1) Problem Definition: : In the robot path problem, a key aspect of navigation is deciding where a robot should move next, considering its present position and the obstacles found in the environment, taking into consideration the pre-assumptions that are fundamental to this problem.

2) Pre-assumptions: When solving robotic path planning, some pre-assumptions and strategies are defined in order to work effectively in a static environment:

- **First assumption**: The location of the robot start point and final destination should be specified.
- **Second assumption**: The movement of the robot is limited to the area of the search space and cannot go beyond the boundaries.
- **Third assumption**: The robot is always moving in the direction to the target point, starting from the start point.
- **Fourth assumption**: In the static environment, each obstacle is placed in a position that has been predefined before running the algorithm. Moreover, all of the obstacles need to be circular in shape with no overlap between them.
- **Fifth assumption**: The RPSO algorithm for path planning will continue to search till the robot reaches a predetermined end point.

B. Robot Path Design Objective Function

The objective function aims to reduce the distance that exists between the robot current point and the end point. The length of the robot's trajectory is effectively quantified by computing this distance. In addressing the challenges in path planning, the primary evaluation criteria is to calculate the length of the path. Consequently, the fitness function is constructed based on these criteria, leading to an objective function that integrates a path length function with its own penalty function that represents constraints.

However, the initial position is denoted as Start (A_0, B_0), and the destination is represented as Goal (A_n, B_n), where each particle represents a sequence of nodal coordinates that constitute a path, which can be denoted as $H = \{ (A_1, B_1), (A_2, B_2), \ldots, (A_n, B_n)\}$ [18].

The objective function is designed to consider both the path's length and the degree to which it adheres to or violates specified constraints, as shown in Eq. 6:

$$f(P_i) = L(P_i) + \beta \cdot V(P_i), \tag{6}$$

Where $L(P_i)$ quantifies the Euclidean distance along the path taken by particle i, $V(P_i)$ represents the extent of constraint violation by the same particle which effectively quantifying how much the path deviates into restricted zones or encounters obstacles. The coefficient β is the penalty factor.

The path length function, denoted as $L(P_i)$ calculates the distance that the mobile robot travels from its starting point ('Start') to its destination ('Goal'), where the point of particles in the search space represents a potential solution to the path planning problem. Moreover, the position of each particle is iteratively updated based on both personal and collective experiences.

For a particle at position $X_i = (x_{i1}, x_{i2}, ..., x_{in})$ and a goal at position $G = (g_1, g_2, ..., g_n)$, the path length function can be defined as shown in Eq.7.

$$L(P_i) = \sqrt{\sum_{k=1}^{n} (x_{ik} - g_n)^2} \tag{7}$$

where n represents the dimensionality of the search space.

The computation of the distance from the particle's position to the goal is shown in Eq.7, where a smaller value of $L(P_i)$ indicates that the particle is closer to the goal, thus representing a potentially better solution. The RPSO algorithm uses this fitness measure to update the velocities and positions of the particles, driving the swarm towards the optimal path solution.

Maintaining a distance between any given path node and an obstacle is a safe threshold, ensuring no intersections with any obstacle, The penalty function $V(P_i)$ emerges as an indispensable constraint to ensure the safety and viability of the robot trajectories. It is designed to assign an elevated P level to paths that exhibit a higher frequency of intersection with obstacles. Within the environmental model, obstacles are

Authorized licensed use limited to: BMS College of Engineering. Downloaded on November 05,2026 at 10:21:27 UTC from IEEE Xplore. Restrictions apply.

represented as a set of circles C_n, each with a center O_n and a number c. To accommodate different size obstacles we apply different radius values $R = \{r_0, r_1, \ldots, r_c\}$ [18].

The mid-node involves the determination of m equidistant points along the segment connecting two consecutive path nodes. Furthermore, the mid-nodes serve as critical points for assessing potential intersections with obstacles.

The Euclidean distance ED is calculated for each node, encompassing both mid-nodes and actual path nodes, with respect to the center of each obstacle O_n, for $k = 1, 2, \ldots, c$. The formula for ED is expressed as illustrated in Eq. 8 [18]:

$$\text{ED} = \sqrt{(A_i - O_{Ak})^2 + (B_i - O_{Bk})^2} \tag{8}$$

Subsequently, the penalty factor P_n for each obstacle C_n is determined based on the following condition:

$$\text{P}_n = \begin{cases} 0, & \text{if ED} \leq r_n \\ 1, & \text{if ED} > r_n \end{cases} \tag{9}$$

where, P_n is set to 1 to impose a penalty whenever the distance that exists between a node in the path and the existing obstacle is lower than the threshold of safety r_n; otherwise, it remains 0.

In addition to finding a path that doesn't have any obstacle, the concept of path smoothing is also important, where sharp directional changes are generally undesirable. The cubic spline is integrated into the RPSO path planning strategy, aiming to enhance the smoothness and efficiency of the robot's movement [18].

Cubic spline interpolation is used for generating smooth curves through piecewise interpolation [19]. It starts with a set of corresponding points $\{A = \{A_0, A_1, \ldots, A_n\}, B = \{B_0, B_1, \ldots, B_n\}\}$. The domain of the spline curve is defined within the interval $[A_{\text{min}}, A_{\text{max}}]$, and $n + 1$ data points within this interval are used to create n subintervals. The cubic spline interpolation function, $S(A)$, must adhere to certain conditions [18], [19]:

- $S(A)$ should be a cubic polynomial within each of the n subintervals $[A_i, A_{i+1}]$ (for $i = 0, 1, \ldots, n-1$).
- It must satisfy $S(A_i) = B_i$ for $i = 0, 1, 2, \ldots, n-1$.
- The first and second derivatives of $S(A)$ should be continuous over the entire interval $[A_{\text{min}}, A_{\text{max}}]$.

To determine the equation of spline interpolation the methodology in [18] is used. According to S(x), which is composed of n cubic polynomials, the polynomial expression can be obtained given by Eq. 10 :

$$S_i(A) = a_i + b_i \cdot (A - A_i) + c_i \cdot (A - A_i)^2 + d_i \cdot (A - A_i)^3 \tag{10}$$

where $a_i, b_i, c_i,$ and d_i (for $i = 0, 1, 2, \ldots, n-1$) are undetermined coefficients, so S(x) has 4n coefficients that are not determined in total.

The RPSO-generated path is denoted as $H = \{ (A_1, B_1), (A_2, B_2), \ldots, (x_n, y_n)\}$, encompassing $n + 2$ nodes, which includes both the start and end points. To smooth this path, cubic spline interpolation is applied over the series $(A_0, x_1, \ldots, A_{n+2})$ and $(B_0, B_1, \ldots, B_{n+2})$. The resulting path is formed by linking lines between adjacent nodes, which comprises the path nodes, points of interpolation, as well as the starting and ending points.

V. Experimental Studies

A. Experimental setup

We performed our experiment in a two-dimensional static environment with obstacles characterized by a distinct set of parameters and constraints that shape the navigation challenges faced by robots. The two-dimensional environment is a representation of all possible states that a robot can search in. where eaxh point is defined primarily by its coordinates (A, B) in a two-dimensional plane. However, in the presence of obstacles, which represent constants in this static world, the robots have to move in a way that avoids these obstacles and guarantees a safe path from their initial starting point to their final end point. These obstacles, which differ in size and form, are represented as circles. Within the configuration of the presumption environment, they establish penalty zones that the robot is not allowed to pass through in order to prevent collisions. The path planning challenge is to find a viable and efficient path from start to goal, navigating around static obstacles while using the reflection strategy to enhance robot path finding.

Moreover, we explore the outcomes of the path design problem, which involves varying the locations of obstacles, N nodes, and iterations, where the experiments included scenarios with various sizes of obstacles. Detailed visualizations and descriptions of each experiment's parameters are provided, along with the optimal costs identified.

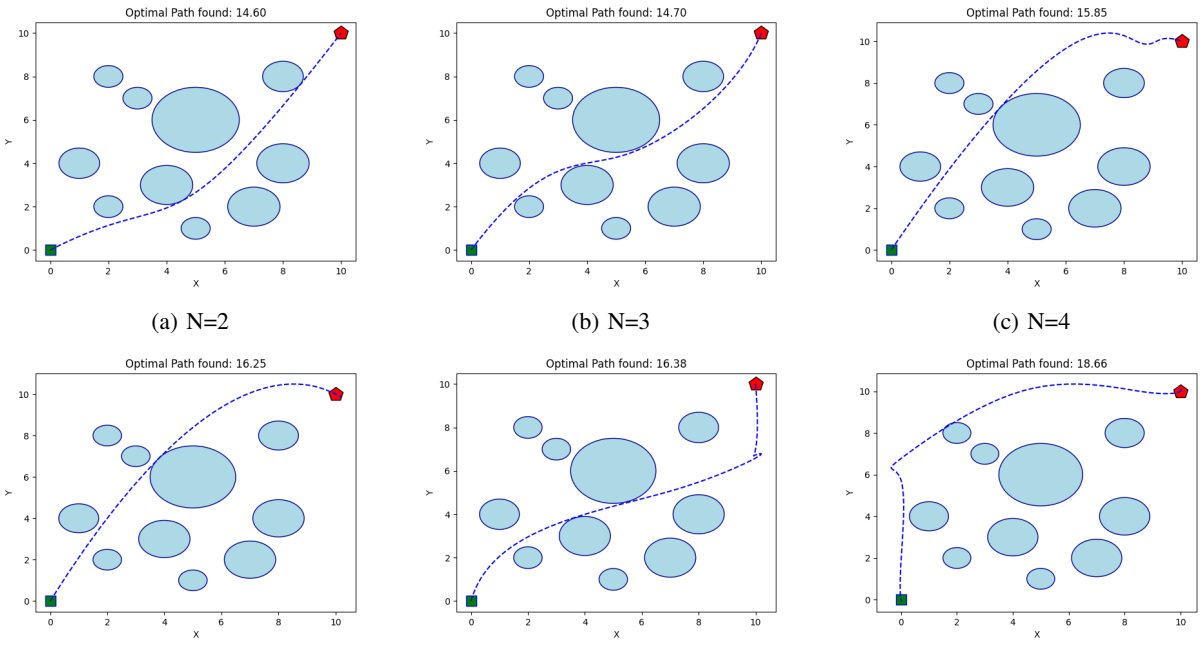
On the other hand, the search space is defined with both A and B ranging from 0 to 10. The starting point is designated at coordinates (0,0), marked by a blue square, while the endpoint is set at (10,10), colored red. The resulting path is depicted with a blue line. The RPSO population size is 50, where the experiment was conducted over a maximum of 100 iterations.

B. Results and Discussion

1) The Impact of Path Node Quantity on Path Planning: To study the impact of path node quantity on path planning, we perform experiments with different numbers of N nodes in the path to investigate how varying the quantity of path nodes affects the effectiveness of path planning. The number of path nodes varying from 1 to 7. The illustrated results are shown in Figure 1, where the paths with different N are displayed.

The results in Figure 1 show the performance of the Reflected Particle Swarm Optimization (RPSO) algorithm in finding optimal paths for a mobile robot in an environment with varying numbers of points (denoted by N). The figure (Fig. 1) illustrates six sub-figures, each corresponding to a different number of points from N=2 to N=7. In each sub-figure, we observe that the RPSO algorithm has successfully generated a path from the start point (marked by a green square) to the end point (marked by a red diamond) while avoiding circular obstacles. The paths are marked by a dashed line.

Authorized licensed use limited to: BMS College of Engineering. Downloaded on November 05,2026 at 10:21:27 UTC from IEEE Xplore. Restrictions apply.



(a) N=2 (b) N=3 (c) N=4

(d) N=5 (e) N=6 (f) N=7

Fig. 1: Result of Reflected PSO at: N=2 to N=7

As N increases, the complexity of the path seems to increase as well. This is expected, as more points generally mean more opportunities for the path to adjust its trajectory around obstacles. The optimal path length increases when the number of points increased. This is a natural consequence of the additional constraints imposed by the points, which require the path to visit intermediate points and thus extend the total distance traveled.

2) The Impact of Different Size and locations of obstacles: Various experiments for the RPSO algorithm with N=3 were conducted to explore different configurations, including varying the locations and sizes of obstacles. The results of these experiments are presented in Fig 2. For each experiment, the optimal path value is calculated. The illustration in Fig. 2 shows that despite the different environments that contain different obstacles of different sizes and locations, RPSO demonstrated impressive efficacy in identifying the optimal path, maintaining its effectiveness even under challenging conditions. This was particularly evident in scenarios where obstacles were positioned near the starting point or when the path was densely populated with large-sized obstacles.

3) Comparison with other optimization algorithms: To effectively evaluate RPSO, we have performed several experiments to compare the algorithm with well-known optimization algorithms, including Particle Swarm Optimization (PSO) [20], Grey Wolf Optimizer (GWO) [21], a hybrid of PSO and GWO (PSOGWO) [22], Sine Cosine Algorithm (SCA) [23], and Moth Flame Optimization (MFO) [24]. The results are shown in Table I and the visualization of the paths is displayed in Fig.3.

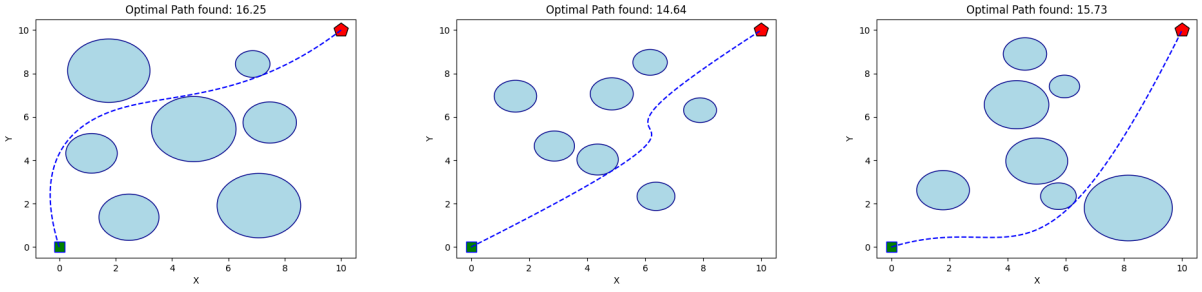
TABLE I: Statistical Measurements of the shortest Path found of RPSO in comparison with other algorithms (30 runs)

Algorithm	Path Mean	MAX path	MIN Path	Std
RPSO	15.62	16.38	14.70	0.59
PSO	16.97	18.54	16.02	0.91
GWO	16.12	17.63	15.21	0.79
PSOGWO	16.72	17.83	15.89	0.72
SCA	20.29	22.54	18.62	1.31
MFO	18.54	20.14	17.25	1.22

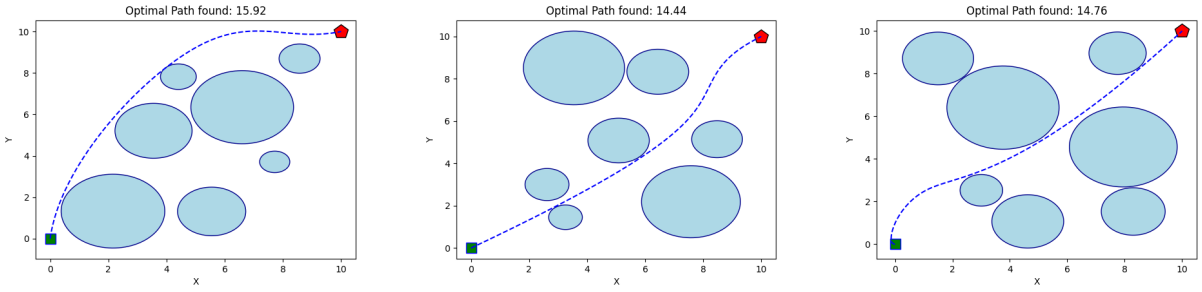
Table I illustrates the comparison results of the RPSO algorithm in robot path planning with other optimization methods. However, the results show that RPSO outperforms the other compared algorithms. In addition, it achieves the lowest mean value, indicating its good performance in path planning by finding the shortest paths across (30) runs. Additionally, RPSO shows the lowest maximum value, suggesting that even in the worst-case scenarios, it tends to outperform most of the other algorithms. This is indicated by its competitive minimum value, showing its ability to find highly efficient paths. Moreover, RPSO exhibits a low standard deviation, which points to its consistent performance across different trials.

Figure 3 shows the comparison of the path planning results between the Reflected Particle Swarm Optimization (RPSO) algorithm and the compared algorithms with N=3. In Fig. 3(a), the RPSO algorithm finds the shortest path among all the algorithms at 14.70 . This indicates that RPSO is effectively utilizing its reflection strategy to find efficient paths that avoid obstacles with minimal length. In contrast, the GWO Fig. 3(b) and PSOGWO Fig. 3(c) find longer paths of 15.67 and 16.59,

Authorized licensed use limited to: BMS College of Engineering. Downloaded on November 05,2026 at 10:21:27 UTC from IEEE Xplore. Restrictions apply.

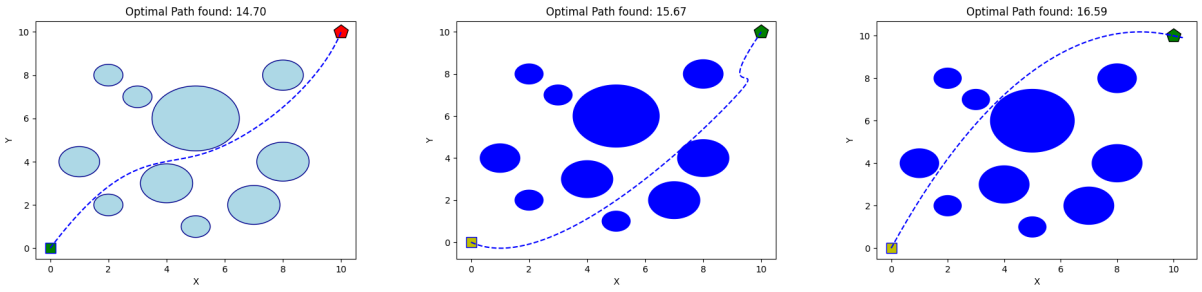


(a) RPSO N=3 (b) RPSO N=3 (c) RPSO N=3

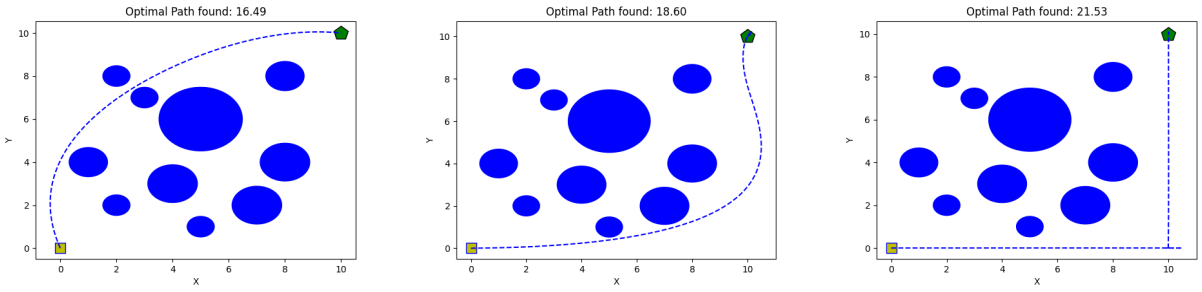


(d) RPSO N=3 (e) RPSO N=3 (f) RPSO N=3

Fig. 2: Results of RPSO with different size and locations of Obstacles



(a) RPSO N=3 (b) GWO N=3 (c) PSOGWO N=3



(d) PSO N=3 (e) MFO N=3 (f) SCA N=3

Fig. 3: Comparison results of RPSO with other optimizers

respectively.

Moreover, the traditional PSO Fig. 3(d) performs similarly to GWO and PSOGWO but does not achieve the efficiency of RPSO, with a path length of 16.49. The MFO Fig. 3(e) results in an even longer path at 18.60. Lastly, the SCA Fig. 3(f) shows the longest path of all at 21.53.

Furthermore, the results show that the superiority of RPSO are evident, where RPSO shows better results than traditional PSO, underscoring the benefits of the reflective mechanism employed in RPSO. Similarly, GWO and PSOGWO, showed competitive results, but did not match the performance levels

Authorized licensed use limited to: BMS College of Engineering. Downloaded on November 05,2026 at 10:21:27 UTC from IEEE Xplore. Restrictions apply.

of RPSO, especially in terms of consistency and best-case scenarios. SCA and MFO with higher mean values and standard deviations, where they appear less effective in consistently finding shorter paths when compared to RPSO.

The RPSO's reflection strategy provide a substantial advantage, allowing the algorithm to explore the search space more effectively and avoid local minima. This is evidenced by the shorter paths found by RPSO in comparison with other algorithms, indicating not only a successful avoidance of obstacles but also an optimization of the path length. The consistency in achieving the shortest path across the experiments for RPSO suggests that its strategy is robust against the different positions and sizes of the obstacles within the given environment.

VI. Conclusion

Reflected Particle Swarm Optimization (RPSO) has been proposed for solving robot path planning in static environment with obstacles. The reflection strategy used in RPSO improves the variety and adaptability of solutions, tackling the issue of convergence and enhancing the algorithm's ability to operate in obstacle-rich areas. RPSO has been testing over a set of experiments with different environments such as varying number of obstacles, size, and numbers of points. It has also been compared with several well-known optimizers, including PSO, GWO, PSOGWO, SCA, and MFO, to evaluate its effectiveness. The findings indicate that RPSO is superior in finding the shortest paths in complex environments and settings. The key to RPSO's effectiveness lies in its reflection method, which simplifies the search area. This approach successfully avoids local optima, leading to the identification of more optimal paths.

References

[1] Prachi Agrawal, Hattan F Abutarboush, Talari Ganesh, and Ali Wagdy Mohamed. Metaheuristic algorithms on feature selection: A survey of one decade of research (2009-2019). *Ieee Access*, 9:26766–26791, 2021.

[2] Xun Li, Dandan Wu, Jingjing He, Muhammad Bashir, and Ma Liping. An improved method of particle swarm optimization for path planning of mobile robot. *Journal of Control Science and Engineering*, 2020, 2020.

[3] Dongshu Wang, Dapei Tan, and Lei Liu. Particle swarm optimization algorithm: an overview. *Soft computing*, 22:387–408, 2018.

[4] Tareq M Shami, Ayman A El-Saleh, Mohammed Alswaitti, Qasem Al-Tashi, Mhd Amen Summakieh, and Seyedali Mirjalili. Particle swarm optimization: A comprehensive survey. *IEEE Access*, 10:10031–10061, 2022.

[5] Mohd Nadhir Ab Wahab, Samia Nefti-Meziani, and Adham Atyabi. A comparative review on mobile robot path planning: Classical or meta-heuristic methods? *Annual Reviews in Control*, 50:233–252, 2020.

[6] Yiying Zhang. Backtracking search algorithm with specular reflection learning for global optimization. *Knowledge-Based Systems*, 212:106546, 2021.

[7] Shiwei Lin, Ang Liu, Jianguo Wang, and Xiaoying Kong. An improved fault-tolerant cultural-pso with probability for multi-agv path planning. *Expert Systems with Applications*, 237, 2024.

[8] Shiwei Lin, Ang Liu, Jianguo Wang, and Xiaoying Kong. An intelligence-based hybrid pso-sa for mobile robot path planning in warehouse. *Journal of Computational Science*, 67, 2023.

[9] Xuezhen Cheng, Jiming Li, Caiyun Zheng, Jianhui Zhang, and Meng Zhao. An improved pso-gwo algorithm with chaos and adaptive inertial weight for robot path planning. *Frontiers in Neurorobotics*, 15, 2021.

[10] Lin Zhang, Yingjie Zhang, and Yangfan Li. Mobile robot path planning based on improved localized particle swarm optimization. *IEEE Sensors Journal*, 21(5):6962 – 6972, 2021.

[11] Hongyan Ma. Application of an improved ant colony algorithm in robot path planning and mechanical arm management. *International Journal of Mechatronics and Applied Mechanics*, 1(10):196 – 203, 2021.

[12] Yan-Jun Shen and Qin-Tian Wang. Mobile robot path planning with two stages based on hybrid intelligent optimisation algorithm. *International Journal of Robotics and Automation*, 38(6):416 – 429, 2023.

[13] Hui Ren, Luli Gao, Xiaochen Shen, Mengnan Li, and Wei Jiang. A novel swarm intelligence algorithm with a parasitism-relation-based structure for mobile robot path planning. *Sensors*, 23(4), 2023.

[14] Syed Fawad Hussain, Adeel Pervez, and Masroor Hussain. Co-clustering optimization using artificial bee colony (abc) algorithm. *Applied Soft Computing*, 97:106725, 2020.

[15] Hao Liu, Xu-Wei Zhang, and Liang-Ping Tu. A modified particle swarm optimization using adaptive strategy. *Expert systems with applications*, 152:113353, 2020.

[16] Zhenhua Yu, Zhijie Si, Xiaobo Li, Dan Wang, and Houbing Song. A novel hybrid particle swarm optimization algorithm for path planning of uavs. *IEEE Internet of Things Journal*, 9(22):22547–22558, 2022.

[17] James Kennedy and Russell Eberhart. Particle swarm optimization. In *Proceedings of ICNN'95-international conference on neural networks*, volume 4, pages 1942–1948. IEEE, 1995.

[18] Qingni Yuan, Ruitong Sun, and Xiaoying Du. Path planning of mobile robots based on an improved particle swarm optimization algorithm. *Processes*, 11(1):26, 2022.

[19] Jianfang Lian, Wentao Yu, Kui Xiao, and Weirong Liu. Cubic spline interpolation-based robot path planning using a chaotic adaptive particle swarm optimization algorithm. *Mathematical problems in engineering*, 2020:1–20, 2020.

[20] Meetu Jain, Vibha Saihjpal, Narinder Singh, and Satya Bir Singh. An overview of variants and advancements of pso algorithm. *Applied Sciences*, 12(17):8392, 2022.

[21] Ganga Negi, Anuj Kumar, Sangeeta Pant, and Mangey Ram. Gwo: a review and applications. *International Journal of System Assurance Engineering and Management*, 12:1–8, 2021.

[22] Fatih Ahmet Şenel, Fatih Gökçe, Asım Sinan Yüksel, and Tuncay Yiğit. A novel hybrid pso–gwo algorithm for optimization problems. *Engineering with Computers*, 35:1359–1373, 2019.

[23] Seyedali Mirjalili. Sca: a sine cosine algorithm for solving optimization problems. *Knowledge-based systems*, 96:120–133, 2016.

[24] Mohammad Shehab, Laith Abualigah, Husam Al Hamad, Hamzeh Alabool, Mohammad Alshinwan, and Ahmad M Khasawneh. Moth–flame optimization algorithm: variants and applications. *Neural Computing and Applications*, 32:9859–9884, 2020.

Authorized licensed use limited to: BMS College of Engineering. Downloaded on November 05,2026 at 10:21:27 UTC from IEEE Xplore. Restrictions apply.